Spam Email Classifier

Machine Learning using TF-IDF and Multinomial Naive Bayes

Internship Project
Student Name: _____
Roll No.: _____

Problem Statement & Objectives

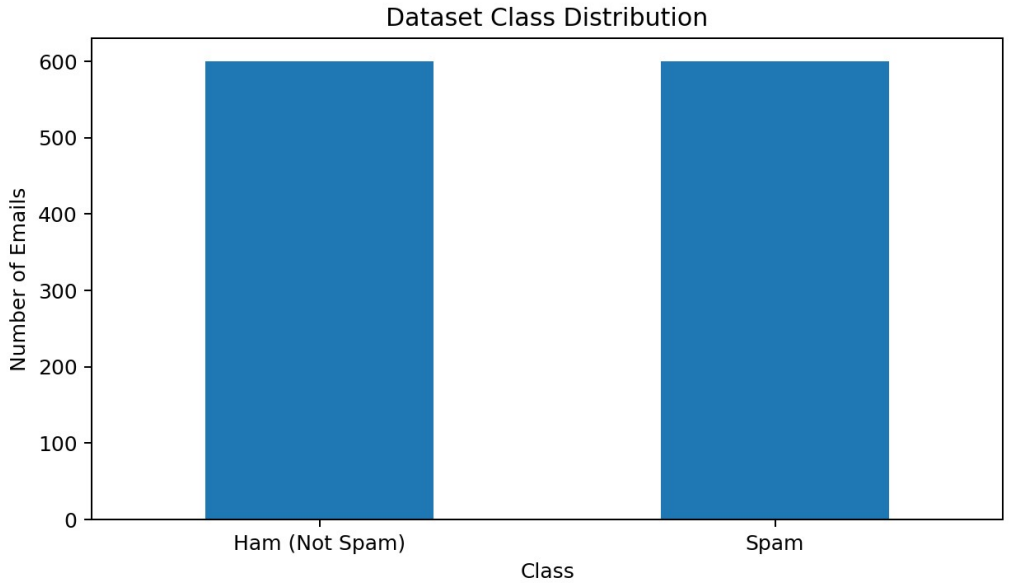
- Classify an incoming email message as spam or ham (not spam).
- Use supervised machine learning instead of only hand-written keyword rules.
- Build a complete, reproducible pipeline: data → features → model → evaluation → prediction.
- Keep the project simple enough for an internship-level demonstration.

System Workflow

The saved pipeline keeps the same TF-IDF transformation attached to the classifier, so new messages are processed consistently.

Dataset & Preprocessing

Bundled offline dataset for reproducible execution



- 1,200 labeled demonstration emails
- 600 spam + 600 ham
- 80% training / 20% testing with stratification
- Lowercase text + English stop-word removal
- Unigram + bigram TF-IDF features

Machine Learning Method

TF-IDF

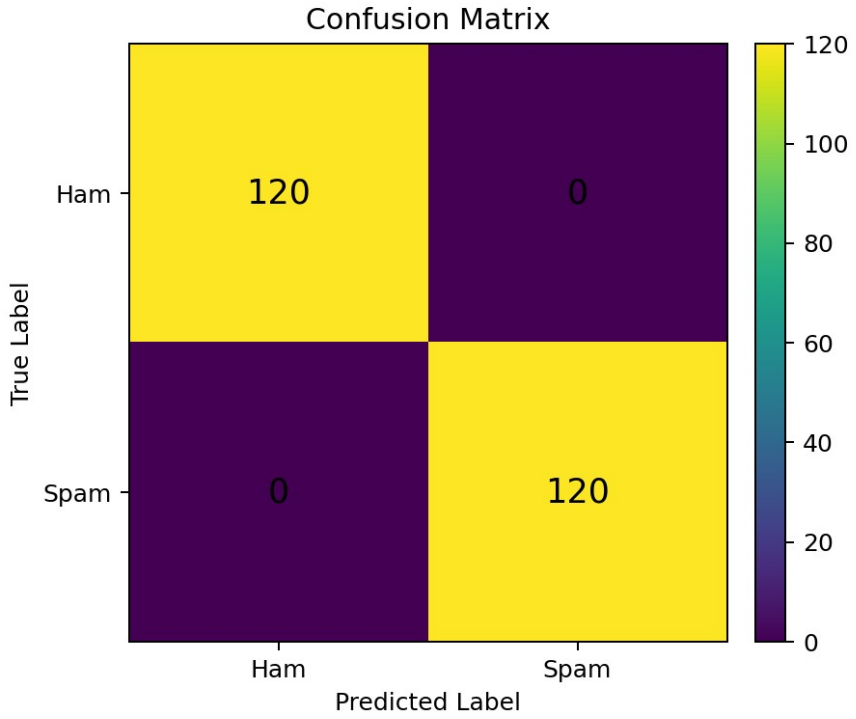
Turns email text into weighted numerical features. Important terms receive stronger weights when they are useful for a document but less common overall.

Multinomial Naive Bayes

Learns class probabilities from the training data. It is fast, simple, and a strong baseline for text classification.

Evaluation Results

20% test set: 240 messages



Accuracy: 100.00%

Precision (spam): 100.00%

Recall (spam): 100.00%

F1-score (spam): 100.00%

Important: the perfect score is specific to the bundled demonstration dataset.

Sample Predictions

Congratulations! You won a free cash prize. Click here to claim it now.

Prediction: SPAM

Hi, please share the meeting notes when you get a chance.

Prediction: HAM

Urgent: verify your account now to avoid suspension.

Prediction: SPAM

The assignment is due tomorrow. Please submit it through the portal.

Prediction: HAM

Project Implementation

- train_model.py — loads data, trains the pipeline, prints metrics, and saves the model.
- app.py — interactive command-line classifier for live demonstration.
- batch_predict.py — classifies every row in a CSV file containing a text column.
- artifacts/spam_classifier.joblib — saved trained pipeline.
- requirements.txt — reproducible Python dependencies.

```
Run:  pip install -r requirements.txt   →   python train_model.py   →   python app.py
```

Limitations & Future Scope

Limitations

- Small template-based demo dataset
- No sender/header/URL features
- New spam patterns can reduce accuracy
- False positives matter in real systems

Future Scope

- Use a large real-world corpus
- Add metadata and URL features
- Compare SVM / Logistic Regression / transformers
- Deploy as a web service and monitor drift

Conclusion

- The project demonstrates a complete NLP classification workflow.
- TF-IDF + Multinomial Naive Bayes provides a compact, fast baseline.
- The included project can be trained and demonstrated offline.
- Real-world deployment would require larger data, broader features, and careful false-positive management.

References: Scikit-learn documentation (TfidfVectorizer, MultinomialNB, Pipeline, metrics)

Thank you